

MongoDB: insta_clone_db Database Commands

Each question is shown in the blue box, followed by its answer. The commands are for mongosh (the current MongoDB shell) and work the same in the older mongo shell.

Question 1

Question 1. Open MongoDB shell and create a new database named `insta_clone_db`, then switch to it using the appropriate commands.

Answer: open a terminal or command prompt and start the shell:

```
mongosh
```

MongoDB has no separate "create database" command. The use command switches to a database, and it is created when data or a collection is first stored in it.

```
use insta_clone_db
```

Output:

```
switched to db insta_clone_db
```

Check which database you are in:

```
db
```

```
insta_clone_db
```

Note: at this point `insta_clone_db` will not yet appear in `show dbs`, because it is still empty. It appears once the first collection is created in Question 2.

Question 2

Question 2. Inside `insta_clone_db`, create a collection called `posts` that will store Instagram-style photo posts.

Answer:

```
db.createCollection("posts")
```

Output:

```
{ ok: 1 }
```

Verify that the collection exists and that the database is now listed:

```
show collections
```

```
show dbs
```

```
posts
```

```
admin          40.00 KiB
config         72.00 KiB
insta_clone_db  8.00 KiB
local          72.00 KiB
```

The sizes will differ on your system. An empty collection is enough to make the database appear.

Optional: a sample post document that fits this collection:

```
db.posts.insertOne({
  username: "riya_travels",
  image_url: "https://example.com/images/post1.jpg",
  caption: "Sunset at the beach",
})
```

```
    likes: 120,  
    hashtags: ["sunset", "beach"]  
  })
```

Question 3

Question 3. Create another collection named `stories` in `insta_clone_db`, and insert a sample story document with fields: `username`, `image_url`, and `timestamp`.

Answer: create the collection explicitly (inserting a document would also create it automatically):

```
db.createCollection("stories")  
  
{ ok: 1 }
```

Insert a sample story. `new Date()` stores the current date and time:

```
db.stories.insertOne({  
  username: "riya_travels",  
  image_url: "https://example.com/images/story1.jpg",  
  timestamp: new Date()  
})
```

Output (your `ObjectId` will be different):

```
{  
  acknowledged: true,  
  insertedId: ObjectId('66f2b4a8e4b0a1b2c3d4e600')  
}
```

Check the inserted story:

```
db.stories.find()  
  
[  
  {  
    _id: ObjectId('66f2b4a8e4b0a1b2c3d4e600'),  
    username: 'riya_travels',  
    image_url: 'https://example.com/images/story1.jpg',  
    timestamp: ISODate('2026-09-24T10:30:00.000Z')  
  }  
]
```

The timestamp will show the date and time when you run the command.

Question 4

Question 4. Drop the `stories` collection from `insta_clone_db` and verify that it no longer exists. Hint: Use the `drop()` method on the collection and show the result with `show collections`.

Answer:

```
db.stories.drop()
```

Output:

```
true
```

`true` means the collection existed and was dropped. If the collection does not exist, `drop()` returns `false`.

Verify:

```
show collections
```

```
posts
```

Only `posts` is listed, so `stories` no longer exists. Another way to check:

```
db.getCollectionNames()  
  
[ 'posts' ]
```

Warning: dropping a collection deletes all its documents permanently.

Question 5

Question 5. Drop the entire `insta_clone_db` database from MongoDB, then use the `show dbs` command to confirm it has been deleted.

Answer: `dropDatabase()` deletes the database you are currently using, so make sure you are in the right one first:

```
use insta_clone_db  
db.dropDatabase()
```

Output (may differ slightly by MongoDB version):

```
switched to db insta_clone_db  
{ ok: 1, dropped: 'insta_clone_db' }
```

Confirm it is gone:

```
show dbs
```

```
admin    40.00 KiB  
config   72.00 KiB  
local    72.00 KiB
```

`insta_clone_db` is no longer in the list, so it has been deleted.

Warning: dropping a database permanently deletes all its collections and data, and it cannot be undone unless you have a backup.

Command Summary

Task	Command
Switch to or create a database	<code>use insta_clone_db</code>
Show current database	<code>db</code>
Create a collection	<code>db.createCollection("posts")</code>
Insert a document	<code>db.stories.insertOne({...})</code>
List collections	<code>show collections</code>
Drop a collection	<code>db.stories.drop()</code>
List databases	<code>show dbs</code>
Drop the current database	<code>db.dropDatabase()</code>